MIST V1.0 Benchmark User Guide

1. The Motivation for Developing This Benchmark

Previous SoC test scheduling datasets, such as the ITC'02 benchmark, only focus on scan tests and do not provide data for test items other than scan tests. In addition, since the ITC'02 benchmark does not offer specific circuits, it is unscalable. This creates the problem of a lack of standard test datasets for researchers aiming to study the joint scheduling of multiple test items, prompting us to construct a custom test dataset.

MIST refers to Multi-Item SoC Test.

2. Important Usage Guidelines

Copyright 2026 School of Microelectronics, Xi'an Jiaotong University, Xi'an 710049, China. All rights reserved.

To facilitate research on test scheduling problems, this benchmark is open for use by all scholars.

When using this benchmark, please cite the paper "AHHTS: An Adaptive Hyper-Heuristic Test Scheduling Framework for Chiplet-Based SoCs", which is the original source of this benchmark.

3. Author Introduction

Qifan Gao received the B.Sc. and M.Sc. degrees from Xi'an Jiaotong University and Tsinghua University, respectively. He is currently pursuing the D.Eng. degree from Xi'an Jiaotong University, Xi'an, China. His main research interests include design-for-test of digital chips and intelligent optimization.

E-mail: 731150921@qq.com.

4. Benchmark Details

4.1. The Origins of Chiplets in the SoC

Due to the semiconductor industry's significant focus on intellectual property protection, nearly all commercial SoCs and chiplets are regarded as core intellectual assets and remain undisclosed to the public. For this reason, well-known open-source chips are employed as the chiplets in the proposed SoC benchmark, as shown in Fig. 1.

				SCAN			MBIST			JTAG		
Circuits	Name	Regs	Gates	Duration*	TAM	power	Duration	TAM	power	Duration	TAM	power
Design1	XuanTieC906	30.5K	183.1K	75	10	224	25	5	101	9	5	3
Design2	Vga_lcd	17.1K	37.3K	26	4	114	6	5	8	5	5	2
Design3	XuanTieE906	304.8K	970.9K	275	8	1505	25	5	77	91	5	28
Design4	Leon2	149.5K	329.0K	176	5	911	38	5	27	45	5	14
Design5	Netcard	96.5K	211.6K	109	6	632	25	5	25	29	5	9
Design6	Leon3mp	109.0K	246.7K	73	6	697	6	5	56	33	5	10
Design7	XuanTieC910	253.1K	1736.3K	287	17	1734	102	5	445	76	5	23
Design8	Leon3-avnet-3s1500	183.8K	387.7K	156	7	1137	13	5	47	55	5	17
Design9	Cva6	24.0K	190.7K	66	14	192	3	5	622	7	5	2
*Duration is measured in milliseconds (ms), TAM is quantified in units, and Power is expressed in milliwatts (mW).												

Fig. 1. Chiplet Information.

It can be seen that the register counts of the selected samples range from 10K to 300K, basically covering the specifications of industrial-grade circuits. Among these circuits, XuanTie is a brand under Alibaba DAMO Academy. Cva6 is an open-source, 6-stage, single-issue, in-order CPU which implements the 64-bit RISC-V instruction set. The remaining circuits are from IWLS05, which is a logic synthesis benchmark suite proposed at the “International Workshop on Logic and Synthesis”.

The method for obtaining each data item in the figure: The power consumption model introduced in [ref 1].The duration and power evaluation method presented in [ref 2].

[ref 1]: R. M. Chou, K. K. Saluja, and V. D. Agrawal, ”Scheduling tests for VLSI systems under power constraints,” IEEE Transactions on Very Large Scale Integration (VLSI) Systems, vol. 5, no. 2, pp. 175-185, 1997, doi:10.1109/92.585217.

[ref 2]: C. P. Ravikumar and B. S. S, ” Test Selection and Scheduling for 3D-Stacked Chiplet-Based Designs,” in 2025 IEEE 9th International Test Conference India (ITC India), 20-22 July 2025, pp. 1-6, doi: 10.1109/ITCIndia66078.2025.11141612.

Download URL:

XuanTieC906:
https://github.com/XUANTIE-RV/openc906

Vga_lcd:
https://iwls.org/iwls2005/benchmarks.html

XuanTieE906:
https://github.com/XUANTIE-RV/opene906

Leon2:
https://iwls.org/iwls2005/benchmarks.html

Netcard:
https://iwls.org/iwls2005/benchmarks.html

Leon3mp:
https://iwls.org/iwls2005/benchmarks.html

XuanTieC910:
https://github.com/XUANTIE-RV/openc910

Leon3-avnet-3s1500:
https://iwls.org/iwls2005/benchmarks.html

Cva6:
https://github.com/openhwgroup/cva6

4.2. Methods for Constructing SoCs Using Chiplets

We refer to the method of constructing SoCs with chiplets described in [ref 3].

[ref 3]: C. Li, L. Deng, L. Qiao, and L. Zhang, ” A constrained multi-objective coevolutionary algorithm with adaptive operator selection for efficient test scheduling in interposer-based 2.5D ICs,” Swarm and Evolutionary Computation, 2025, doi: 10.1016/j.swevo.2025.102085.

We constructed nine SoCs using the aforementioned chiplets. The chiplets included in each SoC are shown in the Fig. 2. Test-mutually exclusive chiplets are also defined in each SoC.

SoCs	Chiplet A	Chiplet B	Chiplet C	Chiplet D	Chiplet E	Mutually Exclusive Group
SoC1	Design 1	Design 2	Design 9	–	–	[Chiplet B, Chiplet C]
SoC2	Design 3	Design 7	Design 8	–	–	[Chiplet B, Chiplet C]
SoC3	Design 4	Design 5	Design 6	Design 8	–	[Chiplet B, Chiplet C]
SoC4	Design 1	Design 2	Design 5	Design 6	–	[Chiplet B, Chiplet C]
SoC5	Design 1	Design 2	Design 3	Design 4	–	[Chiplet B, Chiplet C]
SoC6	Design 5	Design 6	Design 7	Design 9	–	[Chiplet B, Chiplet C]
SoC7	Design 3	Design 4	Design 7	Design 8	–	[Chiplet B, Chiplet C]
SoC8	Design 1	Design 3	Design 5	Design 7	Design 9	[Chiplet B, Chiplet C]
SoC9	Design 1	Design 2	Design 4	Design 6	Design 8	[Chiplet B, Chiplet C]

Fig. 2. SoCs information.

4.3. Data of the Final SoCs.

SoC1:

```
#SoC 1
task_a1 = Task(chiplet_id='A', duration=75, io=10, power=224, task_type='scan')
task_a2 = Task(chiplet_id='A', duration=25, io=5, power=101, task_type='mbist')
task_a3 = Task(chiplet_id='A', duration=9, io=5, power=3, task_type='JTAG')
task_b1 = Task(chiplet_id='B', duration=26, io=4, power=114, task_type='scan')
task_b2 = Task(chiplet_id='B', duration=6, io=5, power=8, task_type='mbist')
task_b3 = Task(chiplet_id='B', duration=5, io=5, power=2, task_type='JTAG')
task_c1 = Task(chiplet_id='C', duration=66, io=14, power=192, task_type='scan')
task_c2 = Task(chiplet_id='C', duration=3, io=5, power=622, task_type='mbist')
task_c3 = Task(chiplet_id='C', duration=7, io=5, power=2, task_type='JTAG')
mutex = ['B', 'C']  # The mutually exclusive list can be customized and is allowed to be empty.
chiplet_a = Chiplet(chiplet_id='A', tasks=[task_a1, task_a2, task_a3])
chiplet_b = Chiplet(chiplet_id='B', tasks=[task_b1, task_b2, task_b3])
chiplet_c = Chiplet(chiplet_id='C', tasks=[task_c1, task_c2, task_c3])
chiplet_based_SoC = [chiplet_a, chiplet_b, chiplet_c]
```

SoC2:

```
#SoC 2
task_a1 = Task(chiplet_id='A', duration=275, io=8, power=1505, task_type='scan')
task_a2 = Task(chiplet_id='A', duration=25, io=5, power=77, task_type='mbist')
task_a3 = Task(chiplet_id='A', duration=91, io=5, power=28, task_type='JTAG')
task_b1 = Task(chiplet_id='B', duration=287, io=17, power=1734, task_type='scan')
task_b2 = Task(chiplet_id='B', duration=102, io=5, power=445, task_type='mbist')
task_b3 = Task(chiplet_id='B', duration=76, io=5, power=23, task_type='JTAG')
task_c1 = Task(chiplet_id='C', duration=156, io=7, power=1137, task_type='scan')
task_c2 = Task(chiplet_id='C', duration=13, io=5, power=47, task_type='mbist')
task_c3 = Task(chiplet_id='C', duration=55, io=5, power=17, task_type='JTAG')
mutex = ['B', 'C'] # The mutually exclusive list can be customized and is allowed to be empty.
chiplet_a = Chiplet(chiplet_id='A', tasks=[task_a1, task_a2, task_a3])
chiplet_b = Chiplet(chiplet_id='B', tasks=[task_b1, task_b2, task_b3])
chiplet_c = Chiplet(chiplet_id='C', tasks=[task_c1, task_c2, task_c3])
chiplet_based_SoC = [chiplet_a, chiplet_b, chiplet_c]
```

SoC3:

```
#SoC 3
task_a1 = Task(chiplet_id='A', duration=176, io=5, power=911, task_type='scan')
task_a2 = Task(chiplet_id='A', duration=38, io=5, power=27, task_type='mbist')
task_a3 = Task(chiplet_id='A', duration=45, io=5, power=14, task_type='JTAG')
task_b1 = Task(chiplet_id='B', duration=109, io=6, power=632, task_type='scan')
task_b2 = Task(chiplet_id='B', duration=25, io=5, power=25, task_type='mbist')
task_b3 = Task(chiplet_id='B', duration=29, io=5, power=9, task_type='JTAG')
task_c1 = Task(chiplet_id='C', duration=73, io=6, power=697, task_type='scan')
task_c2 = Task(chiplet_id='C', duration=6, io=5, power=56, task_type='mbist')
task_c3 = Task(chiplet_id='C', duration=33, io=5, power=10, task_type='JTAG')
task_d1 = Task(chiplet_id='D', duration=156, io=7, power=1137, task_type='scan')
task_d2 = Task(chiplet_id='D', duration=13, io=5, power=47, task_type='mbist')
task_d3 = Task(chiplet_id='D', duration=55, io=5, power=17, task_type='JTAG')
mutex = ['B', 'C'] # The mutually exclusive list can be customized and is allowed to be empty.
chiplet_a = Chiplet(chiplet_id='A', tasks=[task_a1, task_a2, task_a3])
chiplet_b = Chiplet(chiplet_id='B', tasks=[task_b1, task_b2, task_b3])
chiplet_c = Chiplet(chiplet_id='C', tasks=[task_c1, task_c2, task_c3])
chiplet_d = Chiplet(chiplet_id='D', tasks=[task_d1, task_d2, task_d3])
chiplet_based_SoC = [chiplet_a, chiplet_b, chiplet_c, chiplet_d]
```

SoC4:

```
#SoC 4
task_a1 = Task(chiplet_id='A', duration=75, io=10, power=224, task_type='scan')
task_a2 = Task(chiplet_id='A', duration=25, io=5, power=101, task_type='mbist')
task_a3 = Task(chiplet_id='A', duration=9, io=5, power=3, task_type='JTAG')
task_b1 = Task(chiplet_id='B', duration=26, io=4, power=114, task_type='scan')
task_b2 = Task(chiplet_id='B', duration=6, io=5, power=8, task_type='mbist')
task_b3 = Task(chiplet_id='B', duration=5, io=5, power=2, task_type='JTAG')
task_c1 = Task(chiplet_id='C', duration=109, io=6, power=632, task_type='scan')
task_c2 = Task(chiplet_id='C', duration=25, io=5, power=25, task_type='mbist')
task_c3 = Task(chiplet_id='C', duration=29, io=5, power=9, task_type='JTAG')
task_d1 = Task(chiplet_id='D', duration=73, io=6, power=697, task_type='scan')
task_d2 = Task(chiplet_id='D', duration=6, io=5, power=56, task_type='mbist')
task_d3 = Task(chiplet_id='D', duration=33, io=5, power=10, task_type='JTAG')
mutex = ['B', 'C']  # The mutually exclusive list can be customized and is allowed to be empty.
chiplet_a = Chiplet(chiplet_id='A', tasks=[task_a1, task_a2, task_a3])
chiplet_b = Chiplet(chiplet_id='B', tasks=[task_b1, task_b2, task_b3])
chiplet_c = Chiplet(chiplet_id='C', tasks=[task_c1, task_c2, task_c3])
chiplet_d = Chiplet(chiplet_id='D', tasks=[task_d1, task_d2, task_d3])
chiplet_based_SoC = [chiplet_a, chiplet_b, chiplet_c, chiplet_d]
```

SoC5:

```
#SoC 5
task_a1 = Task(chiplet_id='A', duration=75, io=10, power=224, task_type='scan')
task_a2 = Task(chiplet_id='A', duration=25, io=5, power=101, task_type='mbist')
task_a3 = Task(chiplet_id='A', duration=9, io=5, power=3, task_type='JTAG')
task_b1 = Task(chiplet_id='B', duration=26, io=4, power=114, task_type='scan')
task_b2 = Task(chiplet_id='B', duration=6, io=5, power=8, task_type='mbist')
task_b3 = Task(chiplet_id='B', duration=5, io=5, power=2, task_type='JTAG')
task_c1 = Task(chiplet_id='C', duration=275, io=8, power=1505, task_type='scan')
task_c2 = Task(chiplet_id='C', duration=25, io=5, power=77, task_type='mbist')
task_c3 = Task(chiplet_id='C', duration=91, io=5, power=28, task_type='JTAG')
task_d1 = Task(chiplet_id='D', duration=176, io=5, power=911, task_type='scan')
task_d2 = Task(chiplet_id='D', duration=38, io=5, power=27, task_type='mbist')
task_d3 = Task(chiplet_id='D', duration=45, io=5, power=14, task_type='JTAG')
mutex = ['B', 'C']  # The mutually exclusive list can be customized and is allowed to be empty.
chiplet_a = Chiplet(chiplet_id='A', tasks=[task_a1, task_a2, task_a3])
chiplet_b = Chiplet(chiplet_id='B', tasks=[task_b1, task_b2, task_b3])
chiplet_c = Chiplet(chiplet_id='C', tasks=[task_c1, task_c2, task_c3])
chiplet_d = Chiplet(chiplet_id='D', tasks=[task_d1, task_d2, task_d3])
chiplet_based_SoC = [chiplet_a, chiplet_b, chiplet_c, chiplet_d]
```

SoC6:

```
#SoC 6
task_a1 = Task(chiplet_id='A', duration=109, io=6, power=632, task_type='scan')
task_a2 = Task(chiplet_id='A', duration=25, io=5, power=25, task_type='mbist')
task_a3 = Task(chiplet_id='A', duration=29, io=5, power=9, task_type='JTAG')
task_b1 = Task(chiplet_id='B', duration=73, io=6, power=697, task_type='scan')
task_b2 = Task(chiplet_id='B', duration=6, io=5, power=56, task_type='mbist')
task_b3 = Task(chiplet_id='B', duration=33, io=5, power=10, task_type='JTAG')
task_c1 = Task(chiplet_id='C', duration=287, io=17, power=1734, task_type='scan')
task_c2 = Task(chiplet_id='C', duration=102, io=5, power=445, task_type='mbist')
task_c3 = Task(chiplet_id='C', duration=76, io=5, power=23, task_type='JTAG')
task_d1 = Task(chiplet_id='D', duration=66, io=14, power=192, task_type='scan')
task_d2 = Task(chiplet_id='D', duration=3, io=5, power=622, task_type='mbist')
task_d3 = Task(chiplet_id='D', duration=7, io=5, power=2, task_type='JTAG')
mutex = ['B', 'C']  # The mutually exclusive list can be customized and is allowed to be empty.
chiplet_a = Chiplet(chiplet_id='A', tasks=[task_a1, task_a2, task_a3])
chiplet_b = Chiplet(chiplet_id='B', tasks=[task_b1, task_b2, task_b3])
chiplet_c = Chiplet(chiplet_id='C', tasks=[task_c1, task_c2, task_c3])
chiplet_d = Chiplet(chiplet_id='D', tasks=[task_d1, task_d2, task_d3])
chiplet_based_SoC = [chiplet_a, chiplet_b, chiplet_c, chiplet_d]
```

SoC7:

```
#SoC 7
task_a1 = Task(chiplet_id='A', duration=275, io=8, power=1505, task_type='scan')
task_a2 = Task(chiplet_id='A', duration=25, io=5, power=77, task_type='mbist')
task_a3 = Task(chiplet_id='A', duration=91, io=5, power=28, task_type='JTAG')
task_b1 = Task(chiplet_id='B', duration=176, io=5, power=911, task_type='scan')
task_b2 = Task(chiplet_id='B', duration=38, io=5, power=27, task_type='mbist')
task_b3 = Task(chiplet_id='B', duration=45, io=5, power=14, task_type='JTAG')
task_c1 = Task(chiplet_id='C', duration=287, io=17, power=1734, task_type='scan')
task_c2 = Task(chiplet_id='C', duration=102, io=5, power=445, task_type='mbist')
task_c3 = Task(chiplet_id='C', duration=76, io=5, power=23, task_type='JTAG')
task_d1 = Task(chiplet_id='D', duration=156, io=7, power=1137, task_type='scan')
task_d2 = Task(chiplet_id='D', duration=13, io=5, power=47, task_type='mbist')
task_d3 = Task(chiplet_id='D', duration=55, io=5, power=17, task_type='JTAG')
mutex = ['B', 'C']  # The mutually exclusive list can be customized and is allowed to be empty.
chiplet_a = Chiplet(chiplet_id='A', tasks=[task_a1, task_a2, task_a3])
chiplet_b = Chiplet(chiplet_id='B', tasks=[task_b1, task_b2, task_b3])
chiplet_c = Chiplet(chiplet_id='C', tasks=[task_c1, task_c2, task_c3])
chiplet_d = Chiplet(chiplet_id='D', tasks=[task_d1, task_d2, task_d3])
chiplet_based_SoC = [chiplet_a, chiplet_b, chiplet_c, chiplet_d]
```

SoC8:

```
#SoC 8
task_a1 = Task(chiplet_id='A', duration=75, io=10, power=224, task_type='scan')
task_a2 = Task(chiplet_id='A', duration=25, io=5, power=101, task_type='mbist')
task_a3 = Task(chiplet_id='A', duration=9, io=5, power=3, task_type='JTAG')
task_b1 = Task(chiplet_id='B', duration=275, io=8, power=1505, task_type='scan')
task_b2 = Task(chiplet_id='B', duration=25, io=5, power=77, task_type='mbist')
task_b3 = Task(chiplet_id='B', duration=91, io=5, power=28, task_type='JTAG')
task_c1 = Task(chiplet_id='C', duration=109, io=6, power=632, task_type='scan')
task_c2 = Task(chiplet_id='C', duration=25, io=5, power=25, task_type='mbist')
task_c3 = Task(chiplet_id='C', duration=29, io=5, power=9, task_type='JTAG')
task_d1 = Task(chiplet_id='D', duration=287, io=17, power=1734, task_type='scan')
task_d2 = Task(chiplet_id='D', duration=102, io=5, power=445, task_type='mbist')
task_d3 = Task(chiplet_id='D', duration=76, io=5, power=23, task_type='JTAG')
task_e1 = Task(chiplet_id='E', duration=66, io=14, power=192, task_type='scan')
task_e2 = Task(chiplet_id='E', duration=3, io=5, power=622, task_type='mbist')
task_e3 = Task(chiplet_id='E', duration=7, io=5, power=2, task_type='JTAG')
mutex = ['B', 'C']  # The mutually exclusive list can be customized and is allowed to be empty.
chiplet_a = Chiplet(chiplet_id='A', tasks=[task_a1, task_a2, task_a3])
chiplet_b = Chiplet(chiplet_id='B', tasks=[task_b1, task_b2, task_b3])
chiplet_c = Chiplet(chiplet_id='C', tasks=[task_c1, task_c2, task_c3])
chiplet_d = Chiplet(chiplet_id='D', tasks=[task_d1, task_d2, task_d3])
chiplet_e = Chiplet(chiplet_id='E', tasks=[task_e1, task_e2, task_e3])
chiplet_based_SoC = [chiplet_a, chiplet_b, chiplet_c, chiplet_d, chiplet_e]
```

SoC9:

```
#SoC 9
task_a1 = Task(chiplet_id='A', duration=75, io=10, power=224, task_type='scan')
task_a2 = Task(chiplet_id='A', duration=25, io=5, power=101, task_type='mbist')
task_a3 = Task(chiplet_id='A', duration=9, io=5, power=3, task_type='JTAG')
task_b1 = Task(chiplet_id='B', duration=26, io=4, power=114, task_type='scan')
task_b2 = Task(chiplet_id='B', duration=6, io=5, power=8, task_type='mbist')
task_b3 = Task(chiplet_id='B', duration=5, io=5, power=2, task_type='JTAG')
task_c1 = Task(chiplet_id='C', duration=176, io=5, power=911, task_type='scan')
task_c2 = Task(chiplet_id='C', duration=38, io=5, power=27, task_type='mbist')
task_c3 = Task(chiplet_id='C', duration=45, io=5, power=14, task_type='JTAG')
task_d1 = Task(chiplet_id='D', duration=73, io=6, power=697, task_type='scan')
task_d2 = Task(chiplet_id='D', duration=6, io=5, power=56, task_type='mbist')
task_d3 = Task(chiplet_id='D', duration=33, io=5, power=10, task_type='JTAG')
task_e1 = Task(chiplet_id='E', duration=156, io=7, power=1137, task_type='scan')
task_e2 = Task(chiplet_id='E', duration=13, io=5, power=47, task_type='mbist')
task_e3 = Task(chiplet_id='E', duration=55, io=5, power=17, task_type='JTAG')
mutex = ['B', 'C']  # The mutually exclusive list can be customized and is allowed to be empty.
chiplet_a = Chiplet(chiplet_id='A', tasks=[task_a1, task_a2, task_a3])
chiplet_b = Chiplet(chiplet_id='B', tasks=[task_b1, task_b2, task_b3])
chiplet_c = Chiplet(chiplet_id='C', tasks=[task_c1, task_c2, task_c3])
chiplet_d = Chiplet(chiplet_id='D', tasks=[task_d1, task_d2, task_d3])
chiplet_e = Chiplet(chiplet_id='E', tasks=[task_e1, task_e2, task_e3])
chiplet_based_SoC = [chiplet_a, chiplet_b, chiplet_c, chiplet_d, chiplet_e]
```

5. Summary

We hope that researchers can utilize this benchmark to develop superior algorithms and jointly contribute to the advancement of test scheduling research.

Best wishes!

Qifan Gao

2026.5